

Overview

This document specifies a formal framework for a probabilistic inference system designed to determine the most likely hypothesis from a discrete set. It is fundamentally based on the Uncertainty of Thoughts algorithm and functions as an extension upon it, but the document should be comprehensible on its own. The system operates as an interactive agent, employing multiple Large Language Models (LLMs) for knowledge elicitation within a Bayesian framework. Optimal interaction is achieved through an M -step lookahead search algorithm, where the agent selects questions to maximise a reward function based on information gain.

Probabilistic Model

Hypothesis Space ($\mathcal{H}$). The Hypothesis Space is a discrete, mutually exclusive set of potential answers or solutions to a question, denoted by $\mathcal{H} = \{h_1, h_2, \dots, h_n\}$. For example, in the context of a clinical diagnosis, $\mathcal{H}$ would represent the set of possible diseases. **Belief State (B).** The Belief State is a probability distribution over the Hypothesis Space. At any step k , the belief state is a vector B_k , where the i -th element represents the probability of hypothesis h_i , given the evidence gathered so far.

$$B_k = [P(h_1|E_1, \dots, E_k), P(h_2|E_1, \dots, E_k), \dots, P(h_n|E_1, \dots, E_k)]$$

Such that $\sum_{i=1}^n P(h_i|E_1, \dots, E_k) = 1$. **Initial Belief State (B_0).** B_0 is the prior distribution over $\mathcal{H}$ before any evidence has been acquired. While it can be a uniform distribution (e.g., $P(h_i) = \frac{1}{n}$ for all i), a more informed prior can be based on population prevalence or domain knowledge.

A preliminary method:

Collect simple patient background, Age, Race, Past medical History, and prompt LLM to generate priors for illnesses : $P(h_i | \text{Age, Race, Medical history})$, since most priors for diseases are available online, for example <https://www.cdc.gov/diabetes/php/data-research/index.html>, by instructing LLM to reference to sources, priors can be less relied on guessing. A drawback to this approach though, is that not all diseases have these statistics and also not generalizable across different tasks. For example, for trouble shooting tasks, the condition “backgrounds” are unclear across different spectrums

Inference

Evidence (E_k). At step k , a question Q_k is posed, and the user's response constitutes the observed evidence, E_k . **Likelihood Elicitation.** For a given piece of evidence E_k , the system queries an LLM to estimate the likelihood function, $P(E_k|h_i)$, for all $h_i \in \mathcal{H}$. This term represents the probability of observing evidence E_k assuming hypothesis h_i is true. **Uncertainty Scaling.** To deal with LLM probability estimate uncertainties, we obtain a confidence score for each likelihood function, $c_i \in [0, 1]$. The original likelihood is adjusted by pulling it towards the neutral probability of 0.5, based on the confidence multiplier. The adjusted likelihood, $P'(E_k|h_i)$ is calculated as the weighted average:

$$P'(E_k|h_i) = (P(E_k|h_i) \times c_i) + (0.5 \times (1 - c_i))$$

Here, if confidence $c_i = 1$, the adjusted likelihood is equal to the original likelihood, otherwise if $c_i = 0$, the adjusted likelihood becomes 0.5, which has no effect on the hypotheses in the belief update. *Note:* For simplicity, will refer to all adjusted likelihoods as simply $P(E_k|h_i)$, with this update process taken as a given.

Uncertainty scaling is not definite. We can experiment with other strategies and we could look at using beta distributions to model the likelihood as a probability itself. This is characteristic of a Hierarchical Bayesian Model, which we could also look at in more detail since we are quite close to that.

Outline the queries

Belief Update. The belief state is updated using Bayes' theorem. The posterior from step $k - 1$ serves as the prior for step k . This process assumes the conditional independence of evidence, that is, E_k is independent of all previous evidence $\{E_1, \dots, E_{k-1}\}$ given a hypothesis h_i . The unnormalised posterior is first calculated

$$P(h_i \mid E_1, \dots, E_k) \propto P(h_i \mid E_1, \dots, E_{k-1}) \times P(E_k \mid h_i)$$

To derive the normalizer, consider a toy example:

$$\begin{aligned} P(A \mid B, C) &= \frac{P(A, B, C)}{P(B, C)} \\ &= \frac{P(A, C \mid B)P(B)}{P(C \mid B)P(B)} \\ &= \frac{P(A, C \mid B)}{P(C \mid B)} \\ &= \frac{P(C \mid A, B)P(A \mid B)}{P(C \mid B)} \end{aligned}$$

Now let $A = h_i$, $B = E_{\{1:k-1\}}$, $C = E_k$ and pluggin them in we have:

$$P(h_i \mid E_1, \dots, E_k) = \frac{P(E_k \mid h_i, E_1, \dots, E_{k-1})P(h_i \mid E_1, \dots, E_{k-1})}{P(E_k \mid E_1, \dots, E_{k-1})}$$

by our conditional independence assumption:

$$P(h_i \mid E_1, \dots, E_k) = \frac{P(E_k \mid h_i)P(h_i \mid E_1, \dots, E_{k-1})}{P(E_k \mid E_1, \dots, E_{k-1})}$$

The total probability $P(E_k \mid E_1, \dots, E_{k-1}) = \sum_{j=1}^n P(h_j \mid E_1, \dots, E_{k-1}) \times P(E_k \mid h_j)$, and so according to Bayes' theorem,

$$P(h_i \mid E_1, \dots, E_k) = \frac{P(h_i \mid E_1, \dots, E_{k-1}) \times P(E_k \mid h_i)}{P(E_k \mid E_1, \dots, E_{k-1})}$$

Decision Tree

The conversational flow is modeled as a tree consisting of two types of nodes.

- **Evidence Nodes:** Represents a state in the conversation after a specific piece of evidence has been received (e.g., the user answered "Yes"). Each Evidence Node contains a belief state (B_k). The children of evidence nodes are question nodes.
- **Question Nodes:** Represents a point where a question is posed to the user. A question node has a set of children, which are the evidence nodes corresponding to each possible answer.

Optimal Question Selection

At any evidence node, the system performs a lookahead search to determine the optimal next question. This is guided by a reward function that values information gain. **Shannon Entropy.** The uncertainty of a belief state B is quantified by its Shannon Entropy:

$$H(B) = - \sum_{i=1}^n P(h_i) \log_2 P(h_i)$$

Information Gain. The information gain of a question Q is the expected reduction in entropy. Let B_{parent} be the belief state at the parent evidence node, and $\{E_j\}$ be the set of possible answers to question Q .

$$\begin{aligned} \text{IG}(Q) &= H(B_{\text{parent}}) - \sum_j P(E_j | E_1, \dots, E_{k-1}, E_v, \dots, E_{v+x}) H(B_j) \\ &= H(B_{\text{parent}}) - \sum_j \left(\sum_{i=1}^n P(h_i | E_1, \dots, E_{k-1}, E_v, \dots, E_{v+x}) P(E_j | h_i) \right) H(B_j) \end{aligned}$$

Where $E_{\{1:k-1\}}$ are real evidence and $E_{\{v:v+x\}}$ are simulated evidence to depth x of the simulation tree, B_j is the resulting belief state if answer E_j is given, and $P(E_j)$ is the marginal probability of observing that answer.

Reward formulation. To balance information gain with other strategic factors, a reward function is defined.

- **Immediate Reward (R_I):** The immediate reward for asking a question Q is a scaled version of its information gain. To discourage questions where any single branching answer is significantly more probable than the rest, we introduce a penalty term controlled by a hyperparameter $\lambda \geq 0$.

$$R_I(Q) = \frac{\text{IG}(Q)}{1 + \lambda (\max_j (P(E_j | E_1, \dots, E_{k-1}, E_v, \dots, E_{v+x})) - \min_j (P(E_j | E_1, \dots, E_{k-1}, E_v, \dots, E_{v+x})))}$$

- **Accumulated Reward (R_A):** To account for the long-term trajectory, the accumulated reward of a question Q is defined recursively from the root of the conversation tree to its current position.

$$R_A(Q) = R_I(Q) + \begin{cases} R_{A(\text{parent}(Q))} & \text{if parent}(Q) \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

- **Expected Future Reward (R_E):** The expected reward of a question is calculated recursively from the leaves of the simulation tree back to the candidate questions. For a given question Q with possible answer branches leading to subsequent questions $Q'_{j,z}$:

$$R_E(Q) = \begin{cases} R_A(Q) & \text{if } Q \text{ is a leaf of the simulation} \\ \sum_j P(E_j) \left(\frac{1}{\rho_j} \sum_z R_E(Q'_{j,z}) \right) & \text{otherwise} \end{cases}$$

Where $\left(\frac{1}{\rho_j} \sum_z R_E(Q'_{j,z}) \right)$ is the average expected future reward of all ρ_j question nodes that are children of the evidence node of E_j .

The M -step Lookahead Algorithm

At a given evidence node, the optimal next question is determined by a lookahead simulation.

Hyperparameters

- M_q : Maximum number of candidate questions to generate at each node.
- M_e : Maximum number of evidence nodes to consider for each generated question.
- D_{sim} : The maximum depth of one iteration of the lookahead algorithm.
- D_{ask} : The maximum depth of the actual conversation with the user.
- $\tau_{\text{confidence}}$: The probability threshold to consider a hypothesis confirmed.

Procedure

1. **Generate:** From the current evidence node, prompt the LLM to generate up to M_q question nodes with up to M_e answer states.

$P(E_j)$ is currently shorthand for the conditional probability of observing that answer. We need to detail how it can be calculated.

The reward function of UoT is catered to their use case, we need to experiment with other options

$P(E_j)$ is shorthand again, see above

2. **Simulate:** For each candidate question, recursively build a simulation tree up to depth D_{sim} . This involves generating further questions and estimating belief state changes for each potential answer.
3. **Evaluate:** At the lowest question nodes of the simulation tree, calculate the accumulated reward (R_A).
4. **Backpropagate:** Calculate the expected future reward R_E for each question node, moving from the leaves back to the initial candidate questions.
5. **Select:** Choose the candidate question with the highest R_E .
6. **Execute & Iterate:** Pose the selected question to the user. Transition to the evidence node corresponding to their answer and repeat the procedure.
7. **Terminate:** The process terminates when a hypothesis h_i in the belief state exceeds the confidence threshold $\tau_{\text{confidence}}$ or the conversation depth reaches D_{ask} .

In general, we need to better specify how the LLM calls are made and what context they are given